ML Through the Looking Glass

Machine Learning with Lenses in Haskell

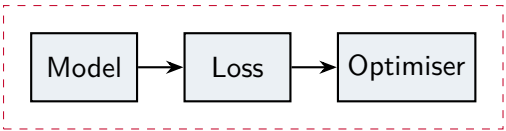
Alexandre Noel

Imperial College London | Supervisor: Prof. Nicolas Wu

MEng Computing (AI & ML)

A statically typed Haskell library for gradient-based learning, built out of optics.

How ML libraries train a model today



a runtime *tape*, rebuilt every step

Three separate components, wired together by hand. The gradients come from a *tape*: every operation is recorded at runtime, then replayed in reverse.[1]

This is convenient, but it has two costs.

1. The tape is rebuilt on every step, so every step pays for graph allocation and dispatch.
2. Shapes are only checked when a kernel runs, so a wrong shape crashes part-way through training:

```
RuntimeError: mat1 and mat2 shapes
cannot be multiplied (8x10 and 4x3)
```

[1] Reverse-mode automatic differentiation, as in PyTorch: Paszke et al., NeurIPS 2019.

What if the model, the loss and the optimiser were one kind of object?

- A layer, a loss and an optimiser could all be the same composable thing.
- The chain rule could be nothing more than composing those things.
- And the whole abstraction could be checked for correctness, then compiled away.

The question this thesis asks

The framework is Cruttwell et al.'s (2021)[2]: category theory, with a dynamically-typed Python PoC. The question is: can we make the pure approach pay in a real language? That is, give up automatic differentiation and gain: speed, no abstraction overhead, and architectural mistakes caught at compile time. At what cost to expressiveness?

[2] Cruttwell, Gavranović, Ghani, Wilson & Zanasi. *Categorical Foundations of Gradient-Based Learning*. arXiv:2103.01931, 2021.

A lens solves the view–update problem

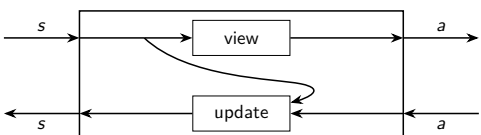
How do you read one part of a structure, and put a modified part back? A *lens* bundles both as one value — here the simplest, same-type variant:

```
data Lens' s a = Lens' { view :: s -> a, update :: s -> a -> s }
```

Focus on the phone number inside an account:

```
newtype PhoneNumber = PhoneNumber Int
data Account = Account
  { phone :: PhoneNumber, accountId :: Int }

acctToPhone :: Lens' Account PhoneNumber
acctToPhone = Lens' phone
                  (\acc n -> acc{ phone = n })
```



view runs forward ($s \to a$); *update* runs backward and needs the original s, so the wire forks. ($s =$ Account, $a =$ Phone.)

One function for both directions — and it composes

Rather than store `view` and `update` separately, the van Laarhoven[3] encoding fuses both directions into a single higher-order function:

$$\texttt{Lens s t a b} = \forall f.\ \text{Functor}\, f \Rightarrow (a \to f\ b) \to s \to f\ t$$

Both directions, from one function. $f = \text{Const}$ recovers the read (view); $f = \text{Identity}$ recovers the write (update).

It composes for free. A lens is just a function in continuation passing style, so two lenses compose with ordinary ($\circ$) — `outer . inner` — no glue. (This is what later collapses the whole stack to direct calls.)

Why four type variables, not $s \to a \to s$? The value put back (b) and the result (t) may differ from what was read.

On the next slide that difference is exactly a *forward value* (b) versus a *backward gradient* (b') — which a simple same-type lens could not express.

[3] The van Laarhoven / profunctor-optics encoding: Pickering, Gibbons & Wu, *Profunctor Optics*, 2017; Kmett's `lens` library.

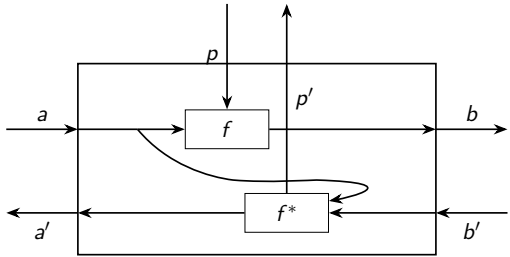
A layer is a lens that also carries parameters

A layer reads its weights going forward and updates them coming back, so we give the parameters a wire of their own:

```
ParaLens p p' a a' b b' = Lens (p,a) (p',a') b b'
```

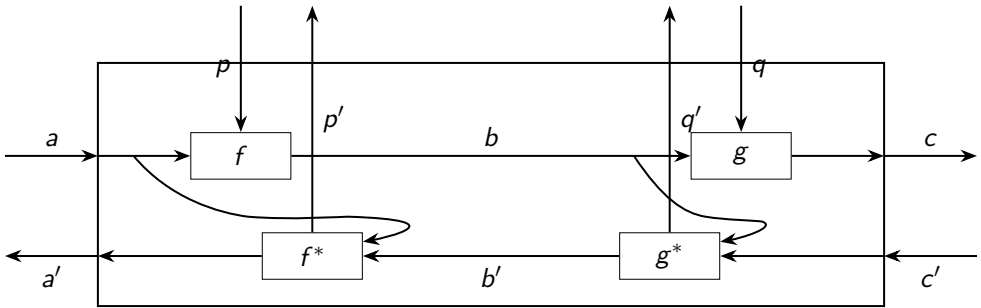
- p, p' — parameters in and out
- a, a' — data forward, gradient back
- b, b' — output forward, gradient in

The forward and backward passes sit in the same value, written and type-checked together. The primed (gradient) types can differ from the forward ones — this is why we needed the type-changing lens from the last slide.



parameters $p \to p'$ on the vertical wire; data $a \to b$ horizontal.

Composing two lenses gives the chain rule, and infers the combined parameters

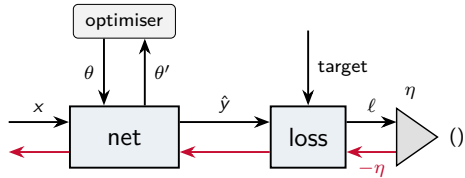


(•) :: `ParaLens p p' a a' b b'` → `ParaLens q q' b b' c c'` → `ParaLens (p,q) (p',q') a a' c c'`

Each layer only defines its own backward pass. Composition assembles the global chain rule, and the combined parameter type (p, q) can be inferred rather than written out.

Every part of the training loop is the same object

- A **layer** is a parametric lens: forward output, backward gradient. `ParaLens' p a b`
- A **loss** is a lens too: forward it outputs the scalar loss; backward it takes the learning rate as the gradient seed. Its parameter is the target. `ParaLens' t pred (t[])`
- The **learning rate** is the *cap* that closes that scalar to the unit `()` — forward discards it, backward injects $-\eta$. `LRLens`
- An **optimiser** is a lens too: it reparametrises the parameter wire (`momentum`, `adam`) via `repara`. `Lens' p p`



The `optimiser` box, at its simplest — plain gradient descent:

```
gradUpdate :: (Num p) => Lens' p p
gradUpdate = lens id (+) -- read p; add
      gradient
```

So one whole training step is a single composed lens, and its backward pass *is* the parameter update.

`(withGradDesc net • loss) ∘ lrSmooth` η `:: ParaLens' (params, target) input ()`

Why build this in (4035 lines of) Haskell?

The lens encoding

A lens is one function, and composing lenses is ordinary function composition. Once the functor is fixed at the call site, it optimises down to direct calls.

no runtime overhead

Kind-level shapes (`DataKinds`)

Shape and batch size are type-level naturals; *type families* compute the conv/pool output shapes. An architectural mismatch does not type-check.

shape errors at compile time

Device & dtype, also in the type

`Tensor dv dt shape` carries the device (CPU/CUDA) and element type (Float/Double) too. One definition runs anywhere, switching is a one-line type application, and an invalid combination is a type error.

one definition, any device & precision

What this work adds beyond the paper

The original is a small dynamically-typed Python proof of concept, storing each lens in an overhead-inducing data structure. This work combines the Cruttwell framework with the *profunctor* (van Laarhoven) optics encoding, the combination behind our zero-cost result, and takes it well past where the paper stops:

- **Static tensor types** (shape, batch, device, dtype) and **batching from the ground up**: vs the original's dynamic types, one example at a time.
- **Zero overhead, verified** by GHC Core.
- **Every backward pass machine-checked** to 10^{-4}, which caught real gradient bugs the prototype had hidden.
- **Fixed two bugs in hasktorch's typed API** to make the conv backward pass type-check at all (Appendix A).
- **Attention**, self- and multi-head[4] are the most complex derivation: a hand-written backward pass with the rank-one softmax Jacobian; head/embedding constraint enforced in the type.
- **Residual** skips as one combinator `skipPara`, where the identity gradient path *falls out* of the structure naturally!
- **Autoencoders** (generative, not only discriminative); **type-level variable depth** (add a layer = one character).

Validated on Iris (93%), MNIST CNN (91%, 1,527 params), residual MNIST, an autoencoder. Modest by design: the correctness evidence is the gradient tests, not the accuracy.

[4] Attention: Vaswani et al., *Attention Is All You Need*, NeurIPS 2017. Residual skips: He et al., *Deep Residual Learning for Image Recognition*, CVPR 2016.

Under the hood: the core is a handful of lines

The framework core — the lens type, sequential composition (•), and the optimiser plug-in:

```
type ParaLens p p' a a' b b' =
  Lens (p,a) (p',a') b b'
type ParaLens' p a b = Lens' (p,a) b

-- sequential composition
(.#.) :: ParaLens p p' a a' b b'
      -> ParaLens q q' b b' c c'
      -> ParaLens (p,q) (p',q') a a' c c'
ab .#. bc = swapFst . rotate
          . rightLens ab . bc

-- plug an optimiser onto the param wire
repara :: Lens q q' p p'
       -> ParaLens p p' a a' b b'
       -> ParaLens q q' a a' b b
repara r = (leftLens r .)
```

A layer is a forward pass and its hand-derived backward, in one value:

```
linear = lens fwd rev where
  fwd (w, x) =
    matmul x (transp w) -- x W^T
  rev (w, x) grad =
    ( matmul (transp grad) x -- dL/dW
    , matmul grad w ) -- dL/dx
```

Forward $xW^\top$; backward the two matrix-calculus gradients. *(T. qualifiers elided.)*

Compose it with the bias via (•) for a full affine layer, the combined parameter can be inferred:

```
-- param inferred as (W, b):
-- (t [o,i], t [o])
matMulLensCore = linear .#. addLens
```

Under the hood: residual connections in one line

A skip connection $y = f(x) + x$ is a single higher-order combinator, built from pieces that already exist:

```
splitIso = iso (join (,)) (uncurry (+)) -- fwd: duplicate; bwd: sum

skipPara f = rightLens splitIso . from rotate . alongside f id . from splitIso
```

`splitIso` fans the input into two copies going forward and *sums* the two gradients coming back; `from splitIso` is its inverse. So the residual's identity gradient path *is* that setter directly! One application is a residual block:

```
resBlock = skipPara (matMulLens . relu .#. matMulLens)
```

Under the hood: depth-`n` networks by typeclass induction

Depth is a compile-time number. `Stack.hs` builds the network — and its parameter type — by induction over Peano naturals:

```
data PeanoNat = One | Succ PeanoNat

type family Stacked n m where -- the parameter type: a nested tuple
  Stacked One m = m
  Stacked (Succ n) m = (m, Stacked n m)

class StackN n where
  stack :: ParaLens' p a a -> ParaLens' (Stacked n p) a a

instance StackN One where stack l = l
instance StackN n => StackN (Succ n) where
  stack l = l .#. stack @n l
  {-# INLINE stack #-} -- crucial: unrolls to zero-overhead Core
```

At each n the parameter type is a fully concrete nested tuple, so GHC unboxes it to the same Core as a hand-written n-layer network.

Under the hood: the attention backward pass, by hand

The reverse pass for scaled dot-product attention: six weight/activation gradients and the softmax correction:

```
rev (p@(wq,wk,wv,wo), x) dOut = ((dWq,dWk,dWv,dWo), dX)
  where (q,k,v, weights, attn, _) = runFwd p x -- recompute intermediates
        dAttn = matmul dOut wo
        dWo = wGrad attn dOut
        dWeights = mm dAttn (tr v)
        dV = mm (tr weights) dAttn
        dScores = softmaxBwd weights dWeights -- the rank-one softmax Jacobian
        dQ = scale' (mm dScores k); dK = scale' (mm (tr dScores) q)
        dWq = wGrad x dQ; dWk = wGrad x dK; dWv = wGrad x dV
        dX = matmul dQ wq + matmul dK wk + matmul dV wv

softmaxBwd w dw = w * sub dw dot -- Y * (dY - sum_j Y_j dY_j)
  where dot = reshape @[b,s,1] (sumDim @2 (w * dw))
```

Live demonstration

Three things to show:

1. the compiler computes the tensor shapes and rejects a bad architecture
2. the lens abstraction vanishes from the compiled code, even at variable depth
3. it trains correctly, and faster than eager PyTorch

Demo, part 1: the compiler does the kernel-size arithmetic (live)

Every spatial dimension in the MNIST CNN is *computed* by a type family ($o = \lfloor (h - k + 2p)/s \rfloor + 1$), not written by hand:

```
[b,1,28,28] --conv 3x3--> [b,3,26,26] --pool--> [b,3,13,13]
          --conv 4x4--> [b,5,10,10] --pool--> [b,5, 5, 5]
          --flatten--> [b,125] --dense 125->10--> [b,10]
```

Change `conv1` from `3x3` to `4x4`: the compiler recomputes the whole chain, the flattened width is no longer 125, and the dense layer stops compiling.

```
  * Couldn't match type '108' with '125' (recomputed conv->pool->flatten width)
```

I wrote none of those intermediate sizes. A wrong kernel is a type error, not a runtime crash.

Demo, part 2: the abstraction vanishes, even at variable depth

One lens network vs a hand-written one: the *same* optimised GHC Core worker.

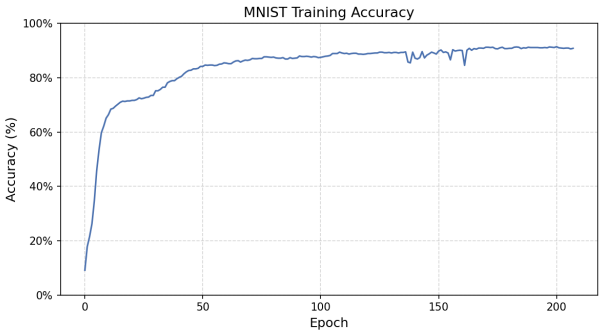
```
$s$wtranspose w1 $wmatmul_tt x W1^T $wadd b1 $wsigmoid_t ...
$s$wtranspose w2 $wmatmul_tt h1 W2^T $wadd b2 $wsigmoid (8 calls, identical)
```

The depth-`n` network is built by typeclass induction over Peano naturals (`Stack.hs`), yet the abstraction is still gone and the Core stays flat:

```
$ grep -c 'ParaLens|repara|stack' Iris.dump-simpl # at @20 and @50
  0
$ wc -l Iris.dump-simpl # @20 -> 1800 @50 -> 4900 (~103/layer, linear)
```

`ParaLens`, (•), `repara`, the van Laarhoven $\forall f$: all absent at every depth. Adding a layer is a one-character change to `@n`.

Demo, part 3: it trains — and the hand-derived gradients are checked



MNIST CNN, 91.4% on the 10k test set (1,527 params, 6k train).

Correctness is not left to chance. Every hand-written backward pass is cross-checked against PyTorch autograd:

`cabal test` $\rightarrow$ `24/24 tests passed` (`max abs err` $< 10^{-4}$)

including the *conv kernel gradient* and the *softmax Jacobian*, plus a full-epoch Iris equivalence test against a dynamic reference.

Demo, part 3 (cont.): faster than eager PyTorch, and why

Implementation (Iris MLP, CPU, per epoch)	Time	
typed lens	**689 μs**	
PyTorch eager (autograd)	3,631 μs	**5.3× slower**

measured with Criterion[5]

There is no tape to build, and the hand-derived backward reduces to the *same* LibTorch calls as the forward. Isolating the cost: building the autograd graph alone is ∼93% of the baseline. The forward pass is within noise of a hand-rolled version (6.9 vs 6.8 μs).

But eager PyTorch is the soft baseline. The honest comparison is JAX →

[5] O'Sullivan, `criterion`: a Haskell micro-benchmarking library.

The honest comparison is JAX

JAX[6] also removes the per-step tape: `jit` traces once and XLA compiles a fused executable. At scale XLA's fusion will *beat* this library (which calls kernels one at a time over FFI, no fusion). **The 5.3× is against eager PyTorch only — not a speed claim against JAX.** The real differences are elsewhere:

	JAX	**This work**
Per-step tape	removed (jit / XLA)	removed (GHC inlining)
Speed at scale	XLA fusion **(wins)**	per-kernel FFI, no fusion
Shape safety	dynamic, at trace, per input	**static, compile-time, total**
Mechanism	tracer → jaxpr → XLA	general compiler, ordinary source
Generality	smooth real-valued autodiff	**any CRDC** (real done; discrete open)

Against JAX the contribution is static safety, general-purpose erasure, and being parametric over the category — not raw speed.

[6] Bradbury et al., *JAX: composable transformations of Python+NumPy programs*, 2018; lowered through the XLA compiler.

What it does not do

Boundaries of the current design:

- Stateful or stochastic layers (dropout, batch-norm) do not fit a pure, deterministic lens.
- Global, step-dependent state has no home: Adam's bias correction needs a step counter, which I left out.
- Attention recomputes its forward pass in the backward; the lens type has no channel for cached activations (and GHC does not spot the duplication here).

Costs of the approach:

- Type errors can be hard to read; a shape mismatch can surface inside a `MonadReader` trace.
- The benchmark is small, CPU-only, and against eager PyTorch; JAX removes the tape too, so this is not a general speed claim.
- The guarantees need Haskell fluency to use.

Where the lens model stops, and the agenda for what comes next.

Where the value is

- Whole classes of configuration mistakes, wrong shapes, wrong device, wrong depth, move from runtime failures to compile errors. Porting the untyped prototype to types actually surfaced gradient bugs that equal-dimension tests had hidden.
- It is a working and efficient instance of the categorical account of learning, rather than only a paper.
- Next steps: discrete and probabilistic CRDCs (learning beyond real vectors), recurrence via `repara` on the state wire, improved inlining of fwd/bwd pass duplication (TTH?).

In short: by writing the backward passes by hand and checking them, and by leaning on the type system and the optimiser, the library gets compile-time safety and no runtime overhead at once.

Thank you for listening, and happy to take questions!

Backup slides

for questions

Backup: van Laarhoven, one function for both directions

Choosing the functor recovers each direction at no cost (`Const` and `Identity` erase via `coerce`[7]):

```
type Lens s t a b = forall f. Functor f => (a -> f b) -> s -> f t
-- f = Const b : fmap = flip const => collapses to the getter (view)
-- f = Identity : fmap = coerce => collapses to the setter (update)
```

That is why composition is just $(\circ)$, and why GHC can inline the whole stack to direct calls.

[7] Safe, zero-cost coercions for Haskell: Breitner, Eisenberg, Peyton Jones & Weirich, ICFP 2014.

Backup: the softmax backward is a rank-one correction

For a row $y = \mathrm{softmax}(x)$ and incoming gradient g:

$$\frac{\partial L}{\partial x_i} \;=\; y_i \left(g_i - \sum_j y_j \, g_j \right).$$

Hand-derived; the inner dot product is one `sumDim` along the class axis, broadcast back. This is the trickiest backward pass in the library, and it is cross-checked against autograd.

Backup: why descent works, the CRDC natural addition

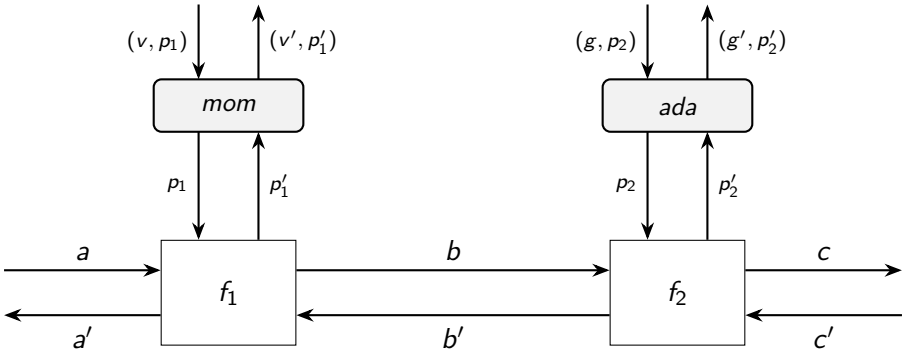
The training loop performs the categorical update[8]

$$\theta_{\text{new}} \;=\; \theta_{\text{old}} + \partial\theta.$$

$\partial\theta$ is whatever the backward pass emits; the sign and scale come from the learning-rate cap ($\alpha = -\eta$ gives descent, $\alpha > 0$ gives ascent, e.g. DeepDream). The update rule itself never changes, and it works in any CRDC, not only Euclidean space.

[8] Cartesian Reverse Differential Categories: Cruttwell et al., arXiv:2103.01931, 2021.

Backup: optimisers as lenses, per layer



f_1 uses momentum, f_2 uses AdaGrad[9]: different optimiser state types on each vertical wire, assembled by $(\bullet)$, with no global optimiser object.

[9] AdaGrad: Duchi, Hazan & Singer, JMLR 2011. Adam: Kingma & Ba, ICLR 2015.

Backup: type-checker and Core scaling with depth

Depth is a compile-time `Nat`; `Stack.hs` builds the network by typeclass induction over Peano naturals:

```
net = runFullModel $ irisModel @20 -- 1,800 lines of Core
net = runFullModel $ irisModel @50 -- 4,900 lines of Core
```

Core grows linearly, roughly 103 lines per layer, and the lens abstraction is absent at every depth. Adding a layer is a one-character change to `@n`.

Backup: why not the `backprop` library?

A van Laarhoven lens is already a single function whose getter is the forward pass and whose setter is the gradient, encoded statically. `backprop`[10], like autograd, builds a runtime tape, which cannot be inlined away and brings back the overhead this design removes. The CRDC update $\theta \leftarrow \theta + \partial\theta$ is also defined for any category, including discrete ones, whereas `backprop` is tied to differentiable real-valued functions.

[10] Le, the `backprop` Haskell library (Hackage), 2018.

Backup: bugs found in hasktorch's typed API (Appendix A)

Getting the convolution backward pass to type-check meant fixing two bugs in the upstream typed bindings[11] (hasktorch 0.2, LibTorch 2.x):

- `convTranspose2d` applied the `ConvSideCheck` constraint with the input and output spatial sizes *swapped*. Invisible for 1×1 kernels (the only case the upstream test suite exercised); for every other kernel it rejected valid calls and accepted invalid ones.
- `im2col`'s binding type-checked but threw at runtime: its `Castable` instance delegated to an unimplemented `makeTuple2`. Fixed by casting each pair through a two-element list instead.

Both surfaced only by pushing the typed API past where it had been exercised — and both are worked around in `Static/Bugfix.hs`, verified against dynamic calls.

[11] hasktorch: typed Haskell bindings to LibTorch, the PyTorch C++ backend.